

Informe Caso Australia

Minería de datos

Caso Australia

En este caso usaremos la metodología CRISP-DM la cual consta de 6 etapas, pero en este informe abarcaremos las primeras dos etapas las que constan de entendimiento del negocio y la comprensión de los datos.

Etapa 1: Entender el negocio

[+ Code](#)[+ Text](#)

La oficina de meteorología de commonwealth en australia nos a dado acceso a este data set para que a travez de tecnicas de computacion como machine learning podamos buscar patrones y poder sacar conclusiones respecto a el clima de Australia. Aqui podemos vizualizar los datos los cuales nos entregaron para su analisis los cuales se hubican en el documento weatherAUS

En esta parte de la primera etapa explicamos el contexto del negocio.

El objetivo es saber si en alguna ciudad lluvio o no al dia siguiente y con la variable de riesgo saber si la cantidad de mm caidos fue un riesgo o no para ese dia.

KPIs

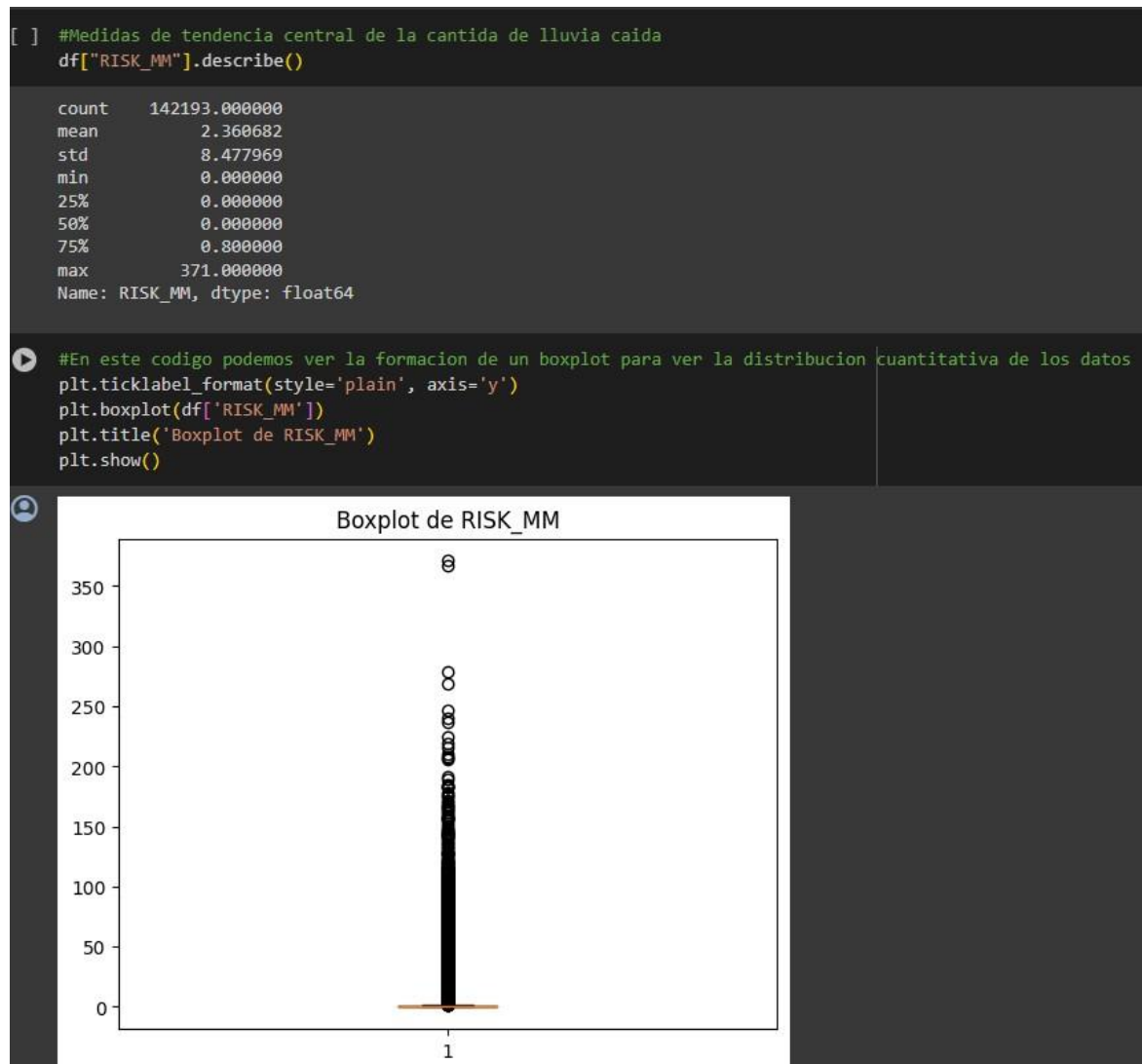
1. En que ciudad caen mas mm de lluvia al dia.
2. Promedio mayor de humedad registrada.
3. En que ciudad se registra la mayor cantidad de horas de sol al dia.
4. En que fecha se registran las mayores precipitaciones en mm.
5. En que ciudad es mas probable que llueva despues de un dia soleado.

En esta parte de la primera etapa damos a conocer los objetivos y los KPIs que tendremos en cuenta durante el desarrollo de este caso.

```
#En esta sentencia se puede apreciar la cantidad de datos nulos que se encuentran en el dataframe especificada por columnas
df.isnull().sum()

Date                0
Location            0
MinTemp            637
MaxTemp            322
Rainfall           1406
Evaporation        60843
Sunshine           67816
WindGustDir        9330
WindGustSpeed      9270
WindDir9am        10013
WindDir3pm        3778
WindSpeed9am       1348
WindSpeed3pm       2630
Humidity9am        1774
Humidity3pm        3610
Pressure9am        14014
Pressure3pm        13981
Cloud9am           53657
Cloud3pm           57094
Temp9am            904
Temp3pm            2726
RainToday          1406
RISK_MM            0
RainTomorrow       0
dtype: int64
```

En esta parte de la segunda etapa podemos apreciar con este código la cantidad de datos nulos que hay por columnas.



En esta parte de la segunda etapa podemos ver el análisis de un dato en específico ya que va acorde con nuestro objetivo y kpis.

En esta parte del informe mostraremos la fase tres y cuatro del caso semestral de minería de datos,

Mostraremos parte de la limpieza de datos además de los distintos modelos que se usaron para el entrenamiento y además de que mostraremos la fase 5 que trata sobre la prueba de estos modelos para llegar a la conclusión de que modelo es más correcto referente a nuestros kpi's.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 142193 entries, 0 to 142192
Data columns (total 26 columns):
 #   Column              Non-Null Count  Dtype  
---  -
 0   Location            142193 non-null float64
 1   MinTemp             142193 non-null float64
 2   MaxTemp             142193 non-null float64
 3   Rainfall            142193 non-null float64
 4   Evaporation         142193 non-null float64
 5   Sunshine            142193 non-null float64
 6   WindGustDir         142193 non-null float64
 7   WindGustSpeed       142193 non-null float64
 8   WindDir9am          142193 non-null float64
 9   WindDir3pm          142193 non-null float64
10   WindSpeed9am        142193 non-null float64
11   WindSpeed3pm        142193 non-null float64
12   Humidity9am         142193 non-null float64
13   Humidity3pm         142193 non-null float64
14   Pressure9am         142193 non-null float64
15   Pressure3pm        142193 non-null float64
16   Cloud9am            142193 non-null float64
17   Cloud3pm            142193 non-null float64
18   Temp9am             142193 non-null float64
19   Temp3pm             142193 non-null float64
20   RainToday           142193 non-null float64
21   RISK_MM             142193 non-null float64
22   RainTomorrow        142193 non-null float64
23   day                 142193 non-null int32  
24   month               142193 non-null int32  
25   year                142193 non-null int32  
dtypes: float64(23), int32(3)
memory usage: 26.6 MB
```

Después de la limpieza de los datos así es como quedaron para su correcto uso en los modelos

Separando la columna de Date que teníamos en tres columnas referente a día mes y año además de rellenar todos los valores nulos que habían en el dataset.

```
0s from sklearn.model_selection import train_test_split

X = df.drop('Rainfall', axis=1) #Aqui borramos la variable objetivo
y = df['Rainfall'] # Aqui asigno la variable objetivo

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=40)

[61] from sklearn.model_selection import train_test_split
df_clasi = df.copy()
X_clasi = df_clasi.drop('RainTomorrow', axis=1) #Aqui borramos la variable objetivo
y_clasi = df_clasi['RainTomorrow'] # Aqui asigno la variable objetivo

X_train1, X_test1, y_train1, y_test1 = train_test_split(X_clasi, y_clasi, test_size=0.20, random_state=40)
```

Como podemos apreciar aquí dividimos la data en train y test en una proporción de 20% y 80% respectivamente los cuales se usará para modelos de regresión, además de hacer una copia del mismo, para poder usarlo en modelos de clasificación.

▼ Fase 4 Modelamiento(Regresion)

Linear Regression

```
[62] from sklearn.linear_model import LinearRegression
modelo = LinearRegression()
modelo.fit(X_train, y_train)
```

LinearRegression

LinearRegression()

Decision Trees

```
from sklearn import tree
modelo2 = tree.DecisionTreeRegressor()
modelo2.fit(X_train, y_train)
```

DecisionTreeRegressor

DecisionTreeRegressor()

Random Forest Regressor

```
[64] from sklearn.ensemble import RandomForestRegressor
modelo3 = RandomForestRegressor(n_estimators=100, random_state=42)
modelo3.fit(X_train, y_train)
```

RandomForestRegressor

RandomForestRegressor(random_state=42)

▼ Fase 4 Modelamiento(Clasificacion)

Random Forest Classifier

✓
8 s

```
from sklearn.ensemble import RandomForestClassifier
modelo4 = RandomForestClassifier(n_estimators=100)
modelo4.fit(X_train1, y_train1)
```



▼ RandomForestClassifier
RandomForestClassifier()

Support Vector Machine (SVM)

✓
2 min

```
[66] from sklearn.svm import SVC
modelo5 = SVC(kernel='linear', probability=True, random_state=42)
modelo5.fit(X_train1, y_train1)
```



▼ SVC
SVC(kernel='linear', probability=True, random_state=42)

En esta imagen podemos apreciar que también se probaron modelos no supervisados para ver qué tan buena es la data que nos dieron

✓ Fase 4 Modelamiento (no supervisado)

```
[ ] df.head()

Location  MinTemp  MaxTemp  Rainfall  Evaporation  Sunshine  WindGustDir  WindGustSpeed  WindDir9am  WindDir3pm  ...  Cloud9am  Cloud3pm  Temp9am  Temp3pm  RainToday  RISK_MM  RainTomorrow  day  month  year
0         1.0     13.4     22.9         0.6         5.47    7.624853    1.000000         44.0         1.0         2.0  ...    8.000000    4.503167     16.9     21.8         0.0         0.0         0.0     1     12    2008
1         1.0       7.4     25.1         0.0         5.47    7.624853    2.000000         44.0         4.0         3.0  ...    4.437189    4.503167     17.2     24.3         0.0         0.0         0.0     2     12    2008
2         1.0     12.9     25.7         0.0         5.47    7.624853    3.000000         46.0         1.0         3.0  ...    4.437189    2.000000     21.0     23.2         0.0         0.0         0.0     3     12    2008
3         1.0       9.2     28.0         0.0         5.47    7.624853    7.955327         24.0        12.0        14.0  ...    4.437189    4.503167     18.1     26.5         0.0         1.0         0.0     4     12    2008
4         1.0     17.5     32.3         1.0         5.47    7.624853    1.000000         41.0         8.0        11.0  ...    7.000000    8.000000     17.8     29.7         0.0         0.2         0.0     5     12    2008

5 rows x 26 columns
```

✓ K-means

```
#Se importa los elementos necesarios para utilizar los clusters,el silhouette_score y pca
from sklearn.cluster import KMeans, DBSCAN
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA

#se crean tres Clustering sobre los datos x_final
kmeans = KMeans(n_clusters=3, random_state=40)
kmeans_labels = kmeans.fit_predict(X_final)

# Calcular Silhouette Score para K-means
silhouette_avg_kmeans = silhouette_score(X_final, kmeans_labels)
print(f"Silhouette Score para K-means: {silhouette_avg_kmeans}")

/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870: FutureWarning: The default value of 'n_init' will change from 10 to 'auto' in 1.4. Set the value of 'n_init' explicitly to suppress the
warnings.warn(
Silhouette Score para K-means: 0.2604142659409414
```

Como podemos apreciar se utilizó K-MEANS , SILHOUTTE_SCORE , DBSCAN además de usar el PCA para achicar la muestra de datos.

✓ DBSCAN

```
#El siguiente codigo busca el mejor resultado para el clustering utilizando para ello el mejor Silhouette Score ademas de ajustarlo al DBSCAN,atravez de un bucles dando los mejores resultados.
import numpy as np

# Función para ajustar DBSCAN y calcular el Silhouette Score
def evaluate_dbscan(X, eps, min_samples):
    dbscan = DBSCAN(eps=eps, min_samples=min_samples)
    dbscan_labels = dbscan.fit_predict(X)

    # Filtrar ruido
    dbscan_labels_filtered = dbscan_labels[dbscan_labels != -1]
    X_filtered = X[dbscan_labels != -1]

    if len(set(dbscan_labels_filtered)) > 1:
        silhouette_avg = silhouette_score(X_filtered, dbscan_labels_filtered)
        return silhouette_avg, dbscan_labels
    else:
        return -1, dbscan_labels

# Ajustar parámetros en un bucle
best_silhouette = -1
best_eps = 0.5
best_min_samples = 5
best_labels = None

eps_values = np.arange(0.2, 1.2, 0.2)
min_samples_values = range(3, 10)

for eps in eps_values:
    for min_samples in min_samples_values:
        silhouette_avg, dbscan_labels = evaluate_dbscan(X_final, eps, min_samples)
        print(f"eps: {eps}, min_samples: {min_samples}, Silhouette Score: {silhouette_avg}")
        if silhouette_avg > best_silhouette:
            best_silhouette = silhouette_avg
            best_eps = eps
            best_min_samples = min_samples
            best_labels = dbscan_labels
```

```
print(f"Mejores parámetros - eps: {best_eps}, min_samples: {best_min_samples}, Mejor Silhouette Score: {best_silhouette}")

eps: 0.2, min_samples: 3, Silhouette Score: -1
eps: 0.2, min_samples: 4, Silhouette Score: -1
eps: 0.2, min_samples: 5, Silhouette Score: -1
eps: 0.2, min_samples: 6, Silhouette Score: -1
eps: 0.2, min_samples: 7, Silhouette Score: -1
eps: 0.2, min_samples: 8, Silhouette Score: -1
eps: 0.2, min_samples: 9, Silhouette Score: -1
eps: 0.4, min_samples: 3, Silhouette Score: -1
eps: 0.4, min_samples: 4, Silhouette Score: -1
eps: 0.4, min_samples: 5, Silhouette Score: -1
eps: 0.4, min_samples: 6, Silhouette Score: -1
eps: 0.4, min_samples: 7, Silhouette Score: -1
eps: 0.4, min_samples: 8, Silhouette Score: -1
eps: 0.4, min_samples: 9, Silhouette Score: -1
eps: 0.6000000000000001, min_samples: 3, Silhouette Score: -1
eps: 0.6000000000000001, min_samples: 4, Silhouette Score: -1
eps: 0.6000000000000001, min_samples: 5, Silhouette Score: -1
eps: 0.6000000000000001, min_samples: 6, Silhouette Score: -1
eps: 0.6000000000000001, min_samples: 7, Silhouette Score: -1
eps: 0.6000000000000001, min_samples: 8, Silhouette Score: -1
eps: 0.6000000000000001, min_samples: 9, Silhouette Score: -1
eps: 0.8, min_samples: 3, Silhouette Score: -1
eps: 0.8, min_samples: 4, Silhouette Score: -1
eps: 0.8, min_samples: 5, Silhouette Score: -1
eps: 0.8, min_samples: 6, Silhouette Score: -1
eps: 0.8, min_samples: 7, Silhouette Score: -1
eps: 0.8, min_samples: 8, Silhouette Score: -1
eps: 0.8, min_samples: 9, Silhouette Score: -1
eps: 1.0, min_samples: 3, Silhouette Score: 0.3264816643053579
eps: 1.0, min_samples: 4, Silhouette Score: -1
eps: 1.0, min_samples: 5, Silhouette Score: -1
eps: 1.0, min_samples: 6, Silhouette Score: -1
eps: 1.0, min_samples: 7, Silhouette Score: -1
eps: 1.0, min_samples: 8, Silhouette Score: -1
eps: 1.0, min_samples: 9, Silhouette Score: -1
Mejores parámetros - eps: 1.0, min_samples: 3, Mejor Silhouette Score: 0.3264816643053579

[ ] # Añadir etiquetas al DataFrame
X_final['KMeans_Labels'] = kmeans_labels
X_final['DBSCAN_Labels'] = dbscan_labels

▼ PCA

• ¿La utilización de PCA es para reducir la dimensionalidad del conjunto de datos separándolos en dos componentes principales, luego crea dataframes con los resultados de PCA y las etiquetas de clustering para finalmente graficar los resultados usando gráficos de dispersión

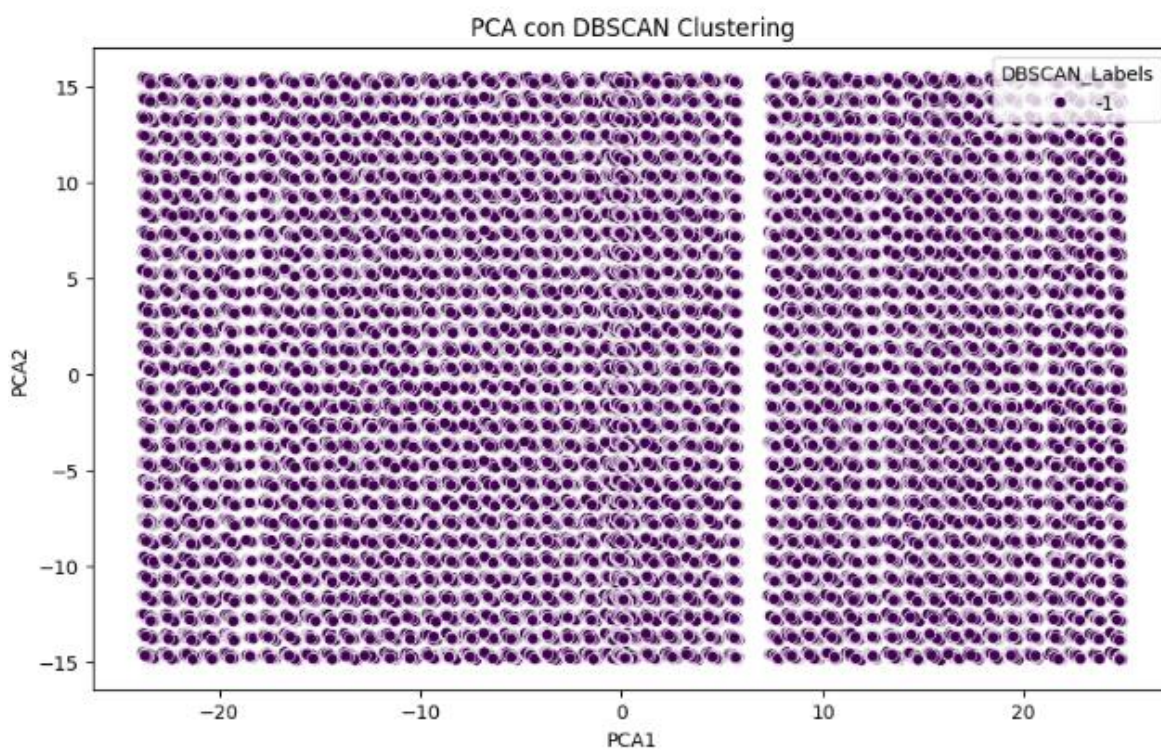
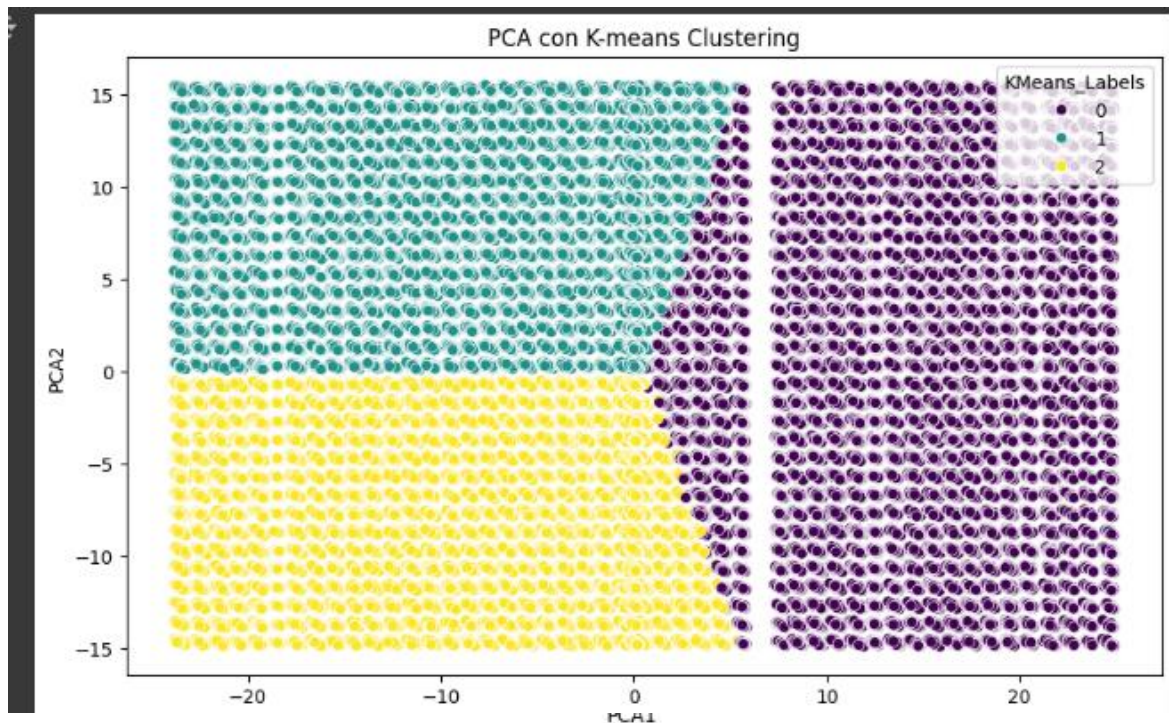
# Solucion PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_final.drop(columns=['KMeans_Labels', 'DBSCAN_Labels']))

# Convertir a dataframe
X_pca_df = pd.DataFrame(X_pca, columns=['PCA1', 'PCA2'])
X_pca_df['KMeans_Labels'] = X_final['KMeans_Labels']
X_pca_df['DBSCAN_Labels'] = X_final['DBSCAN_Labels']

# Graficar los resultados de PCA con K-means
plt.figure(figsize=(10, 6))
sns.scatterplot(data=X_pca_df, x='PCA1', y='PCA2', hue='KMeans_Labels', palette='viridis')
plt.title('PCA con K-means Clustering')
plt.show()

# Graficar los resultados de PCA con DBSCAN
plt.figure(figsize=(10, 6))
sns.scatterplot(data=X_pca_df, x='PCA1', y='PCA2', hue='DBSCAN_Labels', palette='viridis')
plt.title('PCA con DBSCAN Clustering')
plt.show()
```

Y continuación los modelos de PCA



En esta parte del informe mostramos las distintas comprobaciones de los modelos que se realizaron tanto en modelos de regresión como en los modelos de clasificación.

▼ Fase 5 Comprobacion del modelo(Regresion)

```
[ ] from sklearn.metrics import r2_score
    from sklearn.metrics import mean_squared_error
```

Linear Regression

```
[ ] y_pred = modelo.predict(X_test)
```

```
[ ] mse = mean_squared_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)

    print("Error cuadrático medio (MSE):", mse)
    print("Coeficiente de determinación (R2 score):", r2)
```

```
↗ Error cuadrático medio (MSE): 49.66555166046164
  Coeficiente de determinación (R2 score): 0.30083437482594677
```

Decision Trees

```
[ ] y_pred2 = modelo2.predict(X_test)
```

```
[ ] mse = mean_squared_error(y_test, y_pred2)
    r2 = r2_score(y_test, y_pred2)

    print("Error cuadrático medio (MSE):", mse)
    print("Coeficiente de determinación (R2 score):", r2)
```

```
↗ Error cuadrático medio (MSE): 91.19731917437322
  Coeficiente de determinación (R2 score): -0.2838280970008591
```

Random Forest Regressor

```
[ ] y_pred3 = modelo3.predict(X_test)
```

```
[ ] mse = mean_squared_error(y_test, y_pred3)
    r2 = r2_score(y_test, y_pred3)

    print("Error cuadrático medio (MSE):", mse)
    print("Coeficiente de determinación (R2 score):", r2)
```

```
↗ Error cuadrático medio (MSE): 40.66703550096699
  Coeficiente de determinación (R2 score): 0.42751077256946524
```

Como se puede apreciar aquí está la comprobación del modelo de clasificación

▼ Fase 5 Comprobacion del modelo(Clasificacion)

```
[ ] from sklearn.metrics import r2_score
    from sklearn.metrics import mean_squared_error
    from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, roc_auc_score, roc_curve
```

Random Forest Classifier

```
[ ] y_pred4 = modelo4.predict(X_test1)
```

```
[ ] mse = mean_squared_error(y_test1, y_pred4)
    r2 = r2_score(y_test1, y_pred4)

    print("Error cuadrático medio (MSE):", mse)
    print("Coeficiente de determinación (R2 score):", r2)
```

```
➦ Error cuadrático medio (MSE): 0.0
  Coeficiente de determinación (R2 score): 1.0
```

```
[ ] accuracy = accuracy_score(y_test1, y_pred4)
    print("Precisión:", accuracy)
```

```
➦ Precisión: 1.0
```

```
[ ] precision = precision_score(y_test1, y_pred4, average='binary')
    print("Precisión:", precision)
```

```
➦ Precisión: 1.0
```

```
[ ] recall = recall_score(y_test1, y_pred4, average='binary')
    print("Sensibilidad:", recall)
```

```
➦ Sensibilidad: 1.0
```

```
🎮 f1 = f1_score(y_test1, y_pred4, average='binary')
    print("F1-Score:", f1)
```

```
➦ F1-Score: 1.0
```

+ C

```
[ ] y_pred_proba4 = modelo4.predict_proba(X_test1)[:, 1]
```

```
[ ] auc = roc_auc_score(y_test1, y_pred_proba4)

# Graficar la curva ROC
fpr, tpr, _ = roc_curve(y_test1, y_pred_proba4)
plt.plot(fpr, tpr, label="AUC = {:.2f}".format(auc))
plt.xlabel("Tasa de Falsos Positivos")
plt.ylabel("Tasa de Verdaderos Positivos")
plt.legend(loc="best")
plt.show()
```

Aquí está el del support vector machine

Support Vector Machine (SVM)

```
[ ] y_pred5 = modelos.predict(X_test1)
```

```
[ ] mse = mean_squared_error(y_test1, y_pred5)
r2 = r2_score(y_test1, y_pred5)

print("Error cuadrático medio (MSE):", mse)
print("Coeficiente de determinación (R2 score):", r2)
```

```
⇒ Error cuadrático medio (MSE): 0.0
Coeficiente de determinación (R2 score): 1.0
```

```
▶ accuracy = accuracy_score(y_test1, y_pred5)
print("Precisión:", accuracy)
```

```
⇒ Precisión: 1.0
```

```
[ ] precision = precision_score(y_test1, y_pred5, average='binary')
print("Precisión:", precision)
```

```
⇒ Precisión: 1.0
```

```
[ ] recall = recall_score(y_test1, y_pred5, average='binary')
print("Sensibilidad:", recall)
```

```
⇒ Sensibilidad: 1.0
```

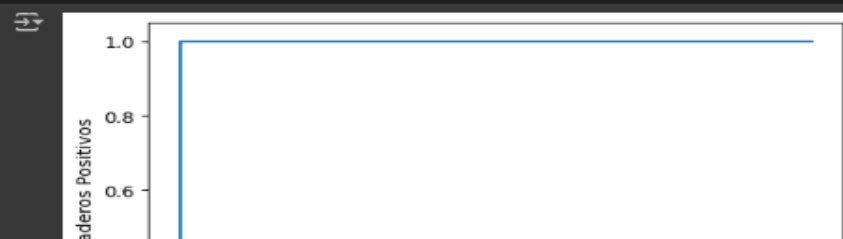
```
[ ] f1 = f1_score(y_test1, y_pred5, average='binary')
print("F1-Score:", f1)
```

```
⇒ F1-Score: 1.0
```

```
[ ] y_pred_proba5 = modelos.predict_proba(X_test1)[: , 1]
```

```
[ ] auc = roc_auc_score(y_test1, y_pred_proba5)

# Graficar la curva ROC
fpr, tpr, _ = roc_curve(y_test1, y_pred_proba5)
plt.plot(fpr, tpr, label="AUC = {:.2f}".format(auc))
plt.xlabel("Tasa de Falsos Positivos")
plt.ylabel("Tasa de Verdaderos Positivos")
plt.legend(loc="best")
plt.show()
```



Fase 6 implementación

```
▼ Fase 6 Deployment

[ ] # Exportar el DataFrame a un archivo CSV
    file_path = '/content/drive/MyDrive/Colab Notebooks/final_model_data.csv'
    df.to_csv(file_path, index=False)

    file_path

📄 '/content/drive/MyDrive/Colab Notebooks/final_model_data.csv'
```

Como se puede apreciar aquí está el código para descargar el csv y poder trabajarlo en power bi

Conclusión

Como conclusión agregar que la data que no pasaron para poder mejorarla a través de las distintas técnicas de machine learning y que gracias a los distintos kpi's nos pudimos guiar a que modelo es mejor para predecir ciertos ámbitos sobre el clima como la lluvia o humedad del ambiente.